SQL Mastery

PJ'S ACADEMY

Complete Course Book

🗄️ SQL Mastery — PJ's Academy

From "what is a database?" to cracking SQL interviews — with 100 practice problems solved in BOTH standard SQL and Snowflake, fully explained.

SQL is the #1 most-requested skill in every data job. This course assumes **zero knowledge** and takes you to interview-ready, with a 100-problem bank (LeetCode-style) where every solution is commented line by line.

🎯 Who This Is For

- Complete beginners (we start from "a table is like a spreadsheet")
- Anyone prepping for data analyst / engineer / scientist interviews
- People who "kind of know SQL" but freeze on window functions and joins

📚 Course Structure

Part	Topic
1	Databases & SELECT basics
2	Filtering, sorting, operators
3	Aggregations & GROUP BY
4	JOINs (the big one)
5	Subqueries & CTEs
6	Window functions
7	Advanced: pivots, recursion, performance
8	**100 Practice Problems** (Easy → Hard)

🧩 The Practice Bank — 100 Problems

The heart of this course. LeetCode-style problems, each with: - 📋 A clear problem statement + sample data - ✅ A **standard SQL** solution (works in MySQL/Postgres) - ❄ A **Snowflake** solution (showing Snowflake-specific idioms like `QUALIFY`) - 💬 **Line-by-line explanation in comments** — so you never get stuck

Tier	Count	File
🟢 Easy	35	easy problems
🟡 Medium	40	medium problems
🔴 Hard	25	hard problems

Practice on the **shared schema** — one set of tables used across all problems.

🖼️ Learn Visually

💡 How To Use This Course

1. Read a Part, run every example.
2. Do the matching practice problems (Easy → Medium → Hard).
3. **Cover the solution, try first, then compare.** That struggle is where learning happens.
4. Redo every problem you missed a week later.

🏆 Outcome

Finish this and you can walk into a SQL interview, handle window functions and multi-table joins calmly, and write clean, correct queries in both standard SQL and Snowflake.

Course: 📘 SQL Mastery — PJ's Academy · hello@pjsacademy.com

🧩 100 SQL Practice Problems — PJ's Academy

LeetCode-style problems. Every solution comes in **standard SQL** AND **Snowflake**, with line-by-line comments explaining the *why*, not just the *what*.

> First, run the shared schema to create the sample tables (employees, departments, customers, products, orders).

🏦 The Bank

Tier	Problems	Focus	File
🟢 **Easy**	1–35	SELECT, WHERE, ORDER BY, basic aggregates, simple joins	01-easy.md
🟡 **Medium**	36–75	Multi-joins, subqueries, CTEs, GROUP BY + HAVING, CASE	02-medium.md
🔴 **Hard**	76–100	Window functions, recursion, top-N-per-group, gaps & islands	03-hard.md

🎯 How To Practice

1. Read the problem. **Try it yourself first** (cover the solution).
2. Compare with the standard-SQL solution + comments.
3. Study the **Snowflake** version — note where it differs (e.g., `QUALIFY`, date functions).
4. Mark problems you missed. **Redo them in a week.**

🧠 SQL vs Snowflake — quick orientation

Most SQL is identical across engines. Snowflake adds power features you'll see used here: - `QUALIFY` — filter window-function results without a subquery (Snowflake gem). - `DATEDIFF`, `DATEADD`, `DATE_TRUNC` — clean date math. - `ILIKE` — case-insensitive matching. - `LISTAGG` — string aggregation. - `::type` — casting shorthand. Where a problem's solution is identical in both, we note "*Snowflake: identical*" and just add any idiomatic tip.

🟢 Easy — Problems 1–35

Foundations: SELECT, WHERE, ORDER BY, LIMIT, basic aggregates, simple joins. Run 00-schema.sql first. Try each yourself before reading the solution.

---

1. Select all employees

Task: Return every column for all employees.

```sql
-- SELECT * returns all columns; FROM names the table
SELECT * FROM employees;
```

❄️ **Snowflake:** identical. 💬 The simplest query. `*` = all columns. In real code, list columns explicitly instead of `*` for clarity and speed.

---

2. Names and salaries only

Task: Return just the name and salary of every employee.

```sql
-- List the specific columns you want, comma-separated
SELECT name, salary FROM employees;
```

❄️ **Snowflake:** identical. 💬 Selecting specific columns is faster and clearer than `*`.

---

3. Employees earning more than 100000

```sql
-- WHERE filters rows; only rows where the condition is TRUE are returned
SELECT name, salary
FROM employees
WHERE salary > 100000;
```

❄️ **Snowflake:** identical. 💬 `WHERE` runs before the columns are returned — it decides *which rows* survive.

---

4. Employees in department 1

```sql
SELECT name FROM employees
WHERE dept_id = 1;    -- = tests exact equality
```

❄️ **Snowflake:** identical.

---

5. Employees NOT in department 2

```sql
SELECT name FROM employees
WHERE dept_id <> 2;   -- <> means "not equal" (!= also works)
```

❄️ **Snowflake:** identical. 💬 ⚠️ Rows where `dept_id` is NULL are **not** returned — NULL comparisons yield UNKNOWN, not TRUE.

---

6. Sort employees by salary (highest first)

```sql
SELECT name, salary FROM employees
ORDER BY salary DESC;   -- DESC = descending; default is ASC
```

❄️ **Snowflake:** identical.

---

7. Top 3 highest-paid employees

```sql
SELECT name, salary FROM employees
ORDER BY salary DESC
LIMIT 3;   -- LIMIT caps the number of rows returned
```

❄️ **Snowflake:** identical (`LIMIT 3` works; `FETCH FIRST 3 ROWS ONLY` also valid). 💬 Sort first, then limit — you get the top 3 by salary.

---

8. Count the employees

```sql
SELECT COUNT(*) AS total_employees
FROM employees;   -- COUNT(*) counts rows; AS renames the output column
```

❄️ **Snowflake:** identical.

---

9. Number of employees with an email

```sql
-- COUNT(column) ignores NULLs, so this counts only non-NULL emails
SELECT COUNT(email) AS with_email
FROM employees;
```

❄️ **Snowflake:** identical. 💬 Key distinction: `COUNT(*)` counts all rows; `COUNT(col)` counts non-NULLs. Nina's NULL email is excluded.

---

10. Highest and lowest salary

```sql
SELECT MAX(salary) AS highest,
       MIN(salary) AS lowest
FROM employees;
```

❄️ **Snowflake:** identical.

---

11. Average salary

```sql
SELECT AVG(salary) AS avg_salary
FROM employees;
```

❄️ **Snowflake:** identical. 💬 `AVG` ignores NULLs. Round it with `ROUND(AVG(salary), 2)`.

---

12. Total payroll

```sql
SELECT SUM(salary) AS total_payroll
FROM employees;
```

❄️ **Snowflake:** identical.

---

13. Employees hired after 2021

```
SELECT name, hire_date FROM employees
WHERE hire_date > '2021-12-31';  -- dates compare like numbers
```

❄️ **Snowflake:** identical. Tip: `WHERE YEAR(hire_date) > 2021` also works in Snowflake.

---

14. Employees hired in 2021

```
SELECT name FROM employees
WHERE hire_date BETWEEN '2021-01-01' AND '2021-12-31';
-- BETWEEN is inclusive on both ends
```

❄️ **Snowflake:** identical, or `WHERE YEAR(hire_date) = 2021`.

---

15. Employees whose name starts with 'A'

```
SELECT name FROM employees
WHERE name LIKE 'A%';   -- % = any sequence of characters
```

❄️ **Snowflake:** identical. Case-insensitive version: `WHERE name ILIKE 'a%'`. 💬 `%` = zero or more chars, `_` = exactly one char.

---

16. Salaries between 80000 and 120000

```
SELECT name, salary FROM employees
WHERE salary BETWEEN 80000 AND 120000;
```

❄️ **Snowflake:** identical.

---

17. Employees in departments 1 or 3

```
SELECT name, dept_id FROM employees
WHERE dept_id IN (1, 3);   -- IN = matches any value in the list
```

❄️ **Snowflake:** identical. 💬 `IN (1,3)` is cleaner than `dept_id = 1 OR dept_id = 3`.

---

18. Employees with no manager (the CEO)

```
SELECT name FROM employees
WHERE manager_id IS NULL;   -- use IS NULL, never = NULL
```

❄️ **Snowflake:** identical. 💬 ⚠️ `= NULL` never works — NULL isn't a value you can equal. Always `IS NULL` / `IS NOT NULL`.

---

19. Distinct departments that have employees

```
SELECT DISTINCT dept_id FROM employees;  -- DISTINCT removes duplicates
```

❄ **Snowflake:** identical.

---

20. Rename salary to monthly (aliasing)

```sql
SELECT name, salary AS monthly_salary
FROM employees;
```

❄ **Snowflake:** identical.

---

21. Employees ordered by department, then salary

```sql
SELECT name, dept_id, salary FROM employees
ORDER BY dept_id ASC, salary DESC;  -- sort by dept, then salary within dept
```

❄ **Snowflake:** identical.

---

22. All customers from Mumbai

```sql
SELECT name FROM customers
WHERE city = 'Mumbai';
```

❄ **Snowflake:** identical (`WHERE city ILIKE 'mumbai'` for case-insensitive).

---

23. Products cheaper than 10000

```sql
SELECT product_name, price FROM products
WHERE price < 10000;
```

❄ **Snowflake:** identical.

---

24. Count products per category

```sql
-- GROUP BY collapses rows sharing a category into one group;
-- COUNT(*) then counts rows within each group
SELECT category, COUNT(*) AS num_products
FROM products
GROUP BY category;
```

❄ **Snowflake:** identical. 💬 Whenever you aggregate "per something", that something goes in `GROUP BY`.

---

25. Average price per category

```sql
SELECT category, ROUND(AVG(price), 2) AS avg_price
FROM products
GROUP BY category;
```

❄ **Snowflake:** identical.

---

26. Number of employees per department

```sql
SELECT dept_id, COUNT(*) AS headcount
FROM employees
GROUP BY dept_id;
```

❄ **Snowflake:** identical.

---

27. Total order amount per customer

```sql
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id;
```

❄ **Snowflake:** identical.

---

28. Departments with more than 1 employee

```sql
SELECT dept_id, COUNT(*) AS headcount
FROM employees
GROUP BY dept_id
HAVING COUNT(*) > 1;    -- HAVING filters GROUPS (WHERE filters rows)
```

❄ **Snowflake:** identical. 💬 `WHERE` filters rows *before* grouping; `HAVING` filters *after* grouping. You can't use aggregates in `WHERE`.

---

29. List employees with their department name (JOIN)

```sql
-- JOIN matches each employee to its department via the shared dept_id
SELECT e.name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

❄ **Snowflake:** identical. 💬 `e` and `d` are table aliases — shorthand so you can write `e.name` instead of `employees.name`.

---

30. Employees and their department location

```sql
SELECT e.name, d.location
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;
```

❄ **Snowflake:** identical.

---

31. Orders with the customer's name

```sql
SELECT o.order_id, c.name, o.amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id;
```

❄ **Snowflake:** identical.

---

32. Orders with product names

```sql
SELECT o.order_id, p.product_name, o.quantity
FROM orders o
JOIN products p ON o.product_id = p.product_id;
```

❄️ **Snowflake:** identical.

---

33. All departments, even those with no employees (LEFT JOIN)

```sql
-- LEFT JOIN keeps every department; employees with no match show as NULL
SELECT d.dept_name, e.name
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id;
```

❄️ **Snowflake:** identical. 💬 A plain `JOIN` (inner) would drop departments with no employees. `LEFT JOIN` keeps the left table's rows regardless.

---

34. Count orders per product (including unordered products)

```sql
SELECT p.product_name, COUNT(o.order_id) AS times_ordered
FROM products p
LEFT JOIN orders o ON p.product_id = o.product_id
GROUP BY p.product_name;
```

❄️ **Snowflake:** identical. 💬 `COUNT(o.order_id)` counts non-NULLs, so unordered products correctly show 0 (not 1).

---

35. Employees earning above the company average

```sql
-- The subquery computes one number (the average); the outer query compares to it
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

❄️ **Snowflake:** identical. 💬 A **scalar subquery** returns a single value you can compare against — your first taste of subqueries (Medium tier goes deeper).

➡️ Continue to 🟡 Medium — Problems 36–75

🟡 Medium — Problems 36–75

Multi-table joins, subqueries, CTEs, CASE, conditional aggregation, anti-joins, date/string logic — the bread and butter of SQL interviews. Run 00-schema.sql first.

---

36. Employees who earn more than their manager

(Classic interview Q — self join)

```sql
-- Join employees to themselves: e = worker, m = their manager
SELECT e.name AS employee, e.salary, m.name AS manager
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

❄️ **Snowflake:** identical. 💬 A **self join** treats one table as two. `m.emp_id = e.manager_id` links each worker to their boss.

---

37. Each employee with their manager's name (show CEO too)

```sql
-- LEFT JOIN so the CEO (manager_id NULL) still appears
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

❄️ **Snowflake:** identical. 💬 With an inner join the CEO would vanish (no manager to match). `LEFT JOIN` keeps them, manager = NULL.

---

38. Customers who never placed an order

(Classic — anti join)

```sql
-- LEFT JOIN then keep only rows with no matching order
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

❄️ **Snowflake:** identical. 💬 The unmatched customers get NULL order columns; filtering `IS NULL` isolates them. Alternative: `WHERE customer_id NOT IN (SELECT customer_id FROM orders)`.

---

39. Same, using NOT EXISTS

```sql
SELECT c.name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

❄️ **Snowflake:** identical. 💬 `NOT EXISTS` is often the fastest anti-join and handles NULLs safely (unlike `NOT IN`).

---

40. Second highest salary

(The most-asked SQL interview question)

```sql
-- Highest salary that is strictly below the maximum
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

❄️ **Snowflake:** identical. (Window-function version appears in Hard #77.) 💬 Robust even with ties. Returns NULL if there's no second value — usually the desired behaviour.

---

41. Departments and their total salary

```sql
SELECT d.dept_name, SUM(e.salary) AS total_salary
FROM departments d
JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

❄️ **Snowflake:** identical.

---

42. Department with the highest average salary

```sql
SELECT dept_id, AVG(salary) AS avg_sal
FROM employees
GROUP BY dept_id
ORDER BY avg_sal DESC
LIMIT 1;
```

❄️ **Snowflake:** identical. 💬 Group → average → sort → take one. (Ties broken arbitrarily; window functions handle ties better — Hard tier.)

---

43. Count employees hired per year

```sql
-- Extract the year, then group by it
SELECT EXTRACT(YEAR FROM hire_date) AS yr, COUNT(*) AS hires
FROM employees
GROUP BY EXTRACT(YEAR FROM hire_date)
ORDER BY yr;
```

❄️ **Snowflake:** `SELECT YEAR(hire_date) AS yr, COUNT(*) ... GROUP BY YEAR(hire_date)` — `YEAR()` is cleaner.

---

44. Label salaries as High / Medium / Low (CASE)

```sql
SELECT name, salary,
  CASE
    WHEN salary >= 120000 THEN 'High'
    WHEN salary >= 80000  THEN 'Medium'
    ELSE 'Low'
  END AS salary_band
FROM employees;
```

❄️ **Snowflake:** identical. 💬 `CASE` is SQL's if/else. Conditions checked top-down; first TRUE wins.

---

45. Count employees in each salary band (CASE + GROUP BY)

```sql
SELECT
  CASE WHEN salary >= 120000 THEN 'High'
       WHEN salary >= 80000  THEN 'Medium'
       ELSE 'Low' END AS band,
  COUNT(*) AS n
FROM employees
GROUP BY
  CASE WHEN salary >= 120000 THEN 'High'
       WHEN salary >= 80000  THEN 'Medium'
       ELSE 'Low' END;
```

❄ **Snowflake:** identical (can also `GROUP BY band` — Snowflake allows grouping by alias).

---

46. Conditional aggregation — count high earners per department

```sql
-- SUM(CASE...) counts rows meeting a condition, per group
SELECT dept_id,
       SUM(CASE WHEN salary >= 100000 THEN 1 ELSE 0 END) AS high_earners,
       COUNT(*) AS total
FROM employees
GROUP BY dept_id;
```

❄ **Snowflake:** identical (or `COUNT_IF(salary >= 100000)` — a Snowflake shortcut). 💬 `SUM(CASE WHEN cond THEN 1 ELSE 0 END)` is the universal "count rows matching a condition within a group" pattern.

---

47. Total revenue per product category

```sql
SELECT p.category, SUM(o.amount) AS revenue
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

❄ **Snowflake:** identical.

---

48. Customers and their total spend, highest first

```sql
SELECT c.name, SUM(o.amount) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name
ORDER BY total_spent DESC;
```

❄ **Snowflake:** identical.

---

49. Customers who spent more than 50000

```sql
SELECT c.name, SUM(o.amount) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name
HAVING SUM(o.amount) > 50000;
```

❄️ **Snowflake:** identical. 💬 Filter on an aggregate → `HAVING`, not `WHERE`.

---

50. Average order value per customer using a CTE

```sql
-- A CTE (WITH) is a named temporary result you can reference below
WITH customer_orders AS (
    SELECT customer_id, COUNT(*) AS num_orders, SUM(amount) AS total
    FROM orders
    GROUP BY customer_id
)
SELECT customer_id, total / num_orders AS avg_order_value
FROM customer_orders;
```

❄️ **Snowflake:** identical. 💬 CTEs make queries readable — build the summary once, use it below.

---

51. Products never ordered

```sql
SELECT product_name FROM products
WHERE product_id NOT IN (SELECT product_id FROM orders);
```

❄️ **Snowflake:** identical. 💬 ⚠️ If the subquery could contain NULLs, `NOT IN` breaks — prefer `NOT EXISTS` there. Here product_id is never NULL, so it's safe.

---

52. Each department's employee count and average salary

```sql
SELECT d.dept_name,
       COUNT(e.emp_id) AS headcount,
       ROUND(AVG(e.salary), 0) AS avg_salary
FROM departments d
LEFT JOIN employees e ON d.dept_id = e.dept_id
GROUP BY d.dept_name;
```

❄️ **Snowflake:** identical. 💬 `LEFT JOIN` + `COUNT(e.emp_id)` → empty departments correctly show 0.

---

53. Orders above the average order amount (correlated idea)

```sql
SELECT order_id, amount FROM orders
WHERE amount > (SELECT AVG(amount) FROM orders);
```

❄️ **Snowflake:** identical.

---

54. Each customer's most recent order date

```sql
SELECT customer_id, MAX(order_date) AS last_order
FROM orders
GROUP BY customer_id;
```

❄️ **Snowflake:** identical.

---

55. Number of distinct products each customer bought

```sql
SELECT customer_id, COUNT(DISTINCT product_id) AS unique_products
FROM orders
GROUP BY customer_id;
```

❄️ **Snowflake:** identical. 💬 `COUNT(DISTINCT ...)` counts unique values — a customer buying the same product twice counts once.

---

56. Combine employee and customer names into one list (UNION)

```sql
-- UNION stacks two result sets and removes duplicates
SELECT name, 'employee' AS type FROM employees
UNION
SELECT name, 'customer' AS type FROM customers;
```

❄️ **Snowflake:** identical. Use `UNION ALL` to keep duplicates (faster — no dedupe).

---

57. Replace NULL email with a placeholder (COALESCE)

```sql
-- COALESCE returns the first non-NULL argument
SELECT name, COALESCE(email, 'no-email@pjs.com') AS email
FROM employees;
```

❄️ **Snowflake:** identical (`IFNULL` and `NVL` also work).

---

58. Employees whose name contains 'a' (case-insensitive)

```sql
SELECT name FROM employees
WHERE LOWER(name) LIKE '%a%';
```

❄️ **Snowflake:** `WHERE name ILIKE '%a%'` — `ILIKE` is built-in case-insensitive.

---

59. Length of each employee's name

```sql
SELECT name, LENGTH(name) AS name_length
FROM employees;
```

❄️ **Snowflake:** identical (`LENGTH` works; `LEN` also).

---

60. Uppercase department names

```sql
SELECT UPPER(dept_name) AS dept_upper FROM departments;
```

❄️ **Snowflake:** identical.

---

61. Extract domain from email

```sql
-- SUBSTRING from the char after '@' to the end
SELECT name,
       SUBSTRING(email FROM POSITION('@' IN email) + 1) AS domain
FROM employees
WHERE email IS NOT NULL;
```

❄️ **Snowflake:** `SPLIT_PART(email, '@', 2) AS domain` — far cleaner. 💬 Learn Snowflake's `SPLIT_PART` — it beats manual substring math.

---

62. Months an employee has been at the company

```sql
SELECT name,
       (EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM hire_date)) * 12
       + (EXTRACT(MONTH FROM CURRENT_DATE) - EXTRACT(MONTH FROM hire_date)) AS months
FROM employees;
```

❄️ **Snowflake:** `DATEDIFF('month', hire_date, CURRENT_DATE) AS months` — one clean function. 💬 Snowflake's `DATEDIFF(part, start, end)` is a huge quality-of-life win over manual date math.

---

63. Orders placed in June 2023

```sql
SELECT order_id, order_date FROM orders
WHERE order_date BETWEEN '2023-06-01' AND '2023-06-30';
```

❄️ **Snowflake:** identical, or `WHERE DATE_TRUNC('month', order_date) = '2023-06-01'`.

---

64. Revenue per month

```sql
SELECT EXTRACT(MONTH FROM order_date) AS mth, SUM(amount) AS revenue
FROM orders
GROUP BY EXTRACT(MONTH FROM order_date)
ORDER BY mth;
```

❄️ **Snowflake:** `SELECT DATE_TRUNC('month', order_date) AS mth, SUM(amount) ... GROUP BY 1` — `DATE_TRUNC` keeps the year too.

---

65. Departments where every employee earns > 60000 (NOT EXISTS)

```sql
SELECT d.dept_name
FROM departments d
WHERE NOT EXISTS (
    SELECT 1 FROM employees e
    WHERE e.dept_id = d.dept_id AND e.salary <= 60000
)
AND EXISTS (SELECT 1 FROM employees e WHERE e.dept_id = d.dept_id);
```

❄️ **Snowflake:** identical. 💬 "All employees satisfy X" = "no employee violates X". The extra `EXISTS` excludes empty departments.

---

66. Top 2 highest-paid employees per... (preview — full version in Hard)

```sql
-- Simple version: overall top 2 (per-group needs window functions, Hard #82)
SELECT name, salary FROM employees
ORDER BY salary DESC
LIMIT 2;
```

❄️ **Snowflake:** identical.

---

67. Customers with more than one order

```sql
SELECT customer_id, COUNT(*) AS order_count
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

❄️ **Snowflake:** identical.

---

68. Average quantity per product category

```sql
SELECT p.category, AVG(o.quantity) AS avg_qty
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category;
```

❄️ **Snowflake:** identical.

---

69. Percentage of total revenue by category

```sql
SELECT p.category,
       SUM(o.amount) AS revenue,
       ROUND(100.0 * SUM(o.amount) / (SELECT SUM(amount) FROM orders), 1) AS pct
FROM orders o
JOIN products p ON o.product_id = p.product_id
GROUP BY p.category;
```

❄️ **Snowflake:** identical. (Window `RATIO_TO_REPORT` does this elegantly — Hard #88.) 💬 `100.0 *` (not `100 *`) forces decimal division — a common gotcha with integer math.

---

70. Employees in the same department as 'Ravi'

```sql
SELECT name FROM employees
WHERE dept_id = (SELECT dept_id FROM employees WHERE name = 'Ravi')
  AND name <> 'Ravi';
```

❄️ **Snowflake:** identical.

---

71. Manager and how many people they manage

```sql
SELECT m.name AS manager, COUNT(e.emp_id) AS reports
FROM employees m
JOIN employees e ON e.manager_id = m.emp_id
GROUP BY m.name
ORDER BY reports DESC;
```

❄️ **Snowflake:** identical.

---

72. Three-table join: customer, product, amount per order

```sql
SELECT o.order_id, c.name AS customer, p.product_name, o.amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN products p  ON o.product_id  = p.product_id;
```

❄️ **Snowflake:** identical. 💬 Chain joins — each `JOIN ... ON` adds one more table on a matching key.

---

73. Categories with revenue above the average category revenue (CTE)

```sql
WITH cat_rev AS (
    SELECT p.category, SUM(o.amount) AS revenue
    FROM orders o JOIN products p ON o.product_id = p.product_id
    GROUP BY p.category
)
SELECT category, revenue FROM cat_rev
WHERE revenue > (SELECT AVG(revenue) FROM cat_rev);
```

❄️ **Snowflake:** identical.

---

74. First and last order date overall, and the span

```sql
SELECT MIN(order_date) AS first_order,
       MAX(order_date) AS last_order
FROM orders;
```

❄️ **Snowflake:** add `DATEDIFF('day', MIN(order_date), MAX(order_date)) AS span_days`.

---

75. Flag whether each employee is above or below company average (CASE + subquery)

```sql
SELECT name, salary,
  CASE WHEN salary > (SELECT AVG(salary) FROM employees)
       THEN 'Above average' ELSE 'Below average' END AS vs_avg
FROM employees;
```

❄️ **Snowflake:** identical.

➡️ Continue to 🔴 Hard — Problems 76–100

🔴 Hard — Problems 76–100

The interview-deciding tier: window functions, ranking, running totals, LAG/LEAD, top-N-per-group, gaps & islands, pivots, and recursion. This is where Snowflake's `QUALIFY` shines. Run 00-schema.sql first.

> A few problems introduce a tiny extra table (given inline) for patterns the base schema can't show (e.g., logins for streaks).

---

76. Rank employees by salary (RANK)

```sql
-- A window function computes across a set of rows "related to" the current row.
-- RANK() OVER (ORDER BY salary DESC) numbers rows by salary; ties share a rank.
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

❄️ **Snowflake:** identical. 💬 `RANK` leaves gaps after ties (1,2,2,4); `DENSE_RANK` doesn't (1,2,2,3); `ROW_NUMBER` is always unique (1,2,3,4).

---

77. Second highest salary (window-function way)

```sql
SELECT DISTINCT salary AS second_highest
FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk = 2;
```

❄️ **Snowflake (cleaner with QUALIFY):**

```sql
SELECT DISTINCT salary
FROM employees
QUALIFY DENSE_RANK() OVER (ORDER BY salary DESC) = 2;
```

💬 `DENSE_RANK = 2` gives the 2nd distinct salary. `QUALIFY` filters window results without a subquery — a Snowflake superpower.

---

78. Nth highest salary (generalise to N=3)

```sql
SELECT DISTINCT salary
FROM (
    SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk = 3;
```

❄️ **Snowflake:** `... QUALIFY DENSE_RANK() OVER (ORDER BY salary DESC) = 3;` 💬 Change the number to get any Nth. Handles ties correctly via `DENSE_RANK`.

---

79. Highest-paid employee in each department (top-1-per-group)

```sql
SELECT dept_id, name, salary
FROM (
    SELECT dept_id, name, salary,
           ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rn
    FROM employees
) t
WHERE rn = 1;
```

❄️ **Snowflake:**

```sql
SELECT dept_id, name, salary
FROM employees
QUALIFY ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) = 1;
```

💬 `PARTITION BY dept_id` restarts the numbering per department — the key idea behind "per group" analytics.

---

80. Salary rank within each department

```sql
SELECT name, dept_id, salary,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank
FROM employees;
```

❄️ **Snowflake:** identical.

---

81. Each employee's salary vs their department average

```sql
-- AVG as a window keeps every row AND shows the group average alongside
SELECT name, dept_id, salary,
       ROUND(AVG(salary) OVER (PARTITION BY dept_id), 0) AS dept_avg,
       salary - AVG(salary) OVER (PARTITION BY dept_id) AS diff
FROM employees;
```

❄️ **Snowflake:** identical. 💬 Unlike `GROUP BY` (which collapses rows), a window aggregate **keeps every row** and attaches the group value — perfect for comparisons.

---

82. Top 2 earners per department

```sql
SELECT dept_id, name, salary
FROM (
    SELECT dept_id, name, salary,
           DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk
    FROM employees
) t
WHERE rnk <= 2;
```

❄️ **Snowflake:** `... QUALIFY DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) <= 2;` 💬 The canonical "top-N-per-group" — memorise this shape; it appears in nearly every SQL interview.

---

83. Running total of revenue by order date

```sql
-- SUM as a window with ORDER BY accumulates row by row
SELECT order_id, order_date, amount,
       SUM(amount) OVER (ORDER BY order_date, order_id
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
       AS running_total
FROM orders;
```

❄️ **Snowflake:** identical. 💬 The frame `UNBOUNDED PRECEDING → CURRENT ROW` = "everything up to and including this row" = a running total.

---

84. Running total of spend per customer

```sql
SELECT customer_id, order_date, amount,
       SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS cust_running
FROM orders;
```

❄️ **Snowflake:** identical. 💬 `PARTITION BY customer_id` resets the accumulation for each customer.

---

85. Month-over-month revenue change (LAG)

```sql
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date) AS mth, SUM(amount) AS revenue
    FROM orders GROUP BY 1
)
SELECT mth, revenue,
       LAG(revenue) OVER (ORDER BY mth) AS prev_month,
       revenue - LAG(revenue) OVER (ORDER BY mth) AS change
FROM monthly;
```

❄️ **Snowflake:** identical (`DATE_TRUNC` is native). 💬 `LAG` grabs the previous row's value — the standard tool for period-over-period comparisons.

---

86. Days between each customer's consecutive orders (LAG on dates)

```sql
SELECT customer_id, order_date,
       LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS prev_order,
       DATEDIFF('day',
                LAG(order_date) OVER (PARTITION BY customer_id ORDER BY order_date),
                order_date) AS days_since_prev
FROM orders;
```

❄️ **Snowflake:** identical. (Standard SQL: `order_date - LAG(order_date) OVER (...)` in Postgres.)

---

87. First and latest order per customer in one row (FIRST_VALUE / LAST_VALUE)

```sql
SELECT DISTINCT customer_id,
       FIRST_VALUE(order_date) OVER (PARTITION BY customer_id ORDER BY order_date) AS first_order,
       LAST_VALUE(order_date)  OVER (PARTITION BY customer_id ORDER BY order_date
            ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_order
FROM orders;
```

❄️ **Snowflake:** identical. 💬 ⚠️ `LAST_VALUE` needs the full-frame clause, else it only sees up to the current row (a classic bug).

---

88. Each category's share of total revenue (RATIO_TO_REPORT)

```sql
SELECT p.category,
       SUM(o.amount) AS revenue,
       ROUND(RATIO_TO_REPORT(SUM(o.amount)) OVER () * 100, 1) AS pct_of_total
FROM orders o JOIN products p ON o.product_id = p.product_id
GROUP BY p.category;
```

❄ **Snowflake:** identical (`RATIO_TO_REPORT` is native). 💬 Standard-SQL fallback: `100.0 * SUM(...) / SUM(SUM(...)) OVER ()` .

---

89. Quartile (NTILE) of employees by salary

```sql
-- NTILE(4) splits rows into 4 roughly equal buckets
SELECT name, salary,
       NTILE(4) OVER (ORDER BY salary DESC) AS salary_quartile
FROM employees;
```

❄ **Snowflake:** identical.

---

90. Cumulative % of employees (running count / total)

```sql
SELECT name, salary,
       ROUND(100.0 * ROW_NUMBER() OVER (ORDER BY salary DESC)
             / COUNT(*) OVER (), 1) AS cumulative_pct
FROM employees;
```

❄ **Snowflake:** identical. 💬 `COUNT(*) OVER ()` (empty OVER) = grand total on every row — handy denominator.

---

91. Median salary

```sql
-- MEDIAN is a built-in ordered aggregate
SELECT MEDIAN(salary) AS median_salary FROM employees;
```

❄ **Snowflake:** identical (`MEDIAN` native; also `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary)`). 💬 Standard SQL (Postgres): `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary)` .

---

92. Pivot: revenue per category as columns (conditional aggregation)

```sql
SELECT
  SUM(CASE WHEN p.category = 'Electronics' THEN o.amount ELSE 0 END) AS electronics,
  SUM(CASE WHEN p.category = 'Furniture'   THEN o.amount ELSE 0 END) AS furniture
FROM orders o JOIN products p ON o.product_id = p.product_id;
```

❄ **Snowflake (native PIVOT):**

```sql
SELECT * FROM (
  SELECT p.category, o.amount FROM orders o JOIN products p ON o.product_id=p.product_id
) PIVOT (SUM(amount) FOR category IN ('Electronics','Furniture'));
```

💬 Conditional aggregation is the portable pivot; Snowflake's `PIVOT` is the native shortcut.

---

93. Consecutive login days per user — the "islands" problem

(Add a small table for this classic)

```sql
-- logins(user_id, login_date). Find streaks of consecutive days.
-- Trick: (date - ROW_NUMBER days) is constant within a consecutive run
WITH ranked AS (
  SELECT user_id, login_date,
         DATEADD('day',
                 -ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY login_date),
                 login_date) AS grp
  FROM logins
)
SELECT user_id, MIN(login_date) AS streak_start,
       MAX(login_date) AS streak_end, COUNT(*) AS streak_len
FROM ranked
GROUP BY user_id, grp
ORDER BY user_id, streak_start;
```

❄ **Snowflake:** identical (`DATEADD` native). Postgres: `login_date - ROW_NUMBER() OVER(...) * INTERVAL '1 day'` . 💬 **Gaps-and-islands**: subtracting a row counter from a date makes consecutive dates map to the same constant — group by it to find streaks. A top interview pattern.

---

94. Find the gaps — missing IDs in a sequence

```sql
-- Where the next order_id jumps by more than 1, there's a gap
SELECT order_id + 1 AS gap_start,
       next_id - 1 AS gap_end
FROM (
  SELECT order_id,
         LEAD(order_id) OVER (ORDER BY order_id) AS next_id
  FROM orders
) t
WHERE next_id - order_id > 1;
```

❄ **Snowflake:** identical. 💬 `LEAD` looks at the next row; a jump > 1 reveals missing IDs.

---

95. Employees earning the same salary as someone else (duplicates via window)

```sql
SELECT name, salary
FROM (
    SELECT name, salary,
           COUNT(*) OVER (PARTITION BY salary) AS same_sal_count
    FROM employees
) t
WHERE same_sal_count > 1;
```

❄ **Snowflake:** `... QUALIFY COUNT(*) OVER (PARTITION BY salary) > 1;` 💬 A partitioned `COUNT(*)` tags every row with how many share its salary — filter > 1 to find duplicates.

---

96. Rank customers by spend and keep only the top 3 (dense)

```sql
WITH spend AS (
    SELECT customer_id, SUM(amount) AS total
    FROM orders GROUP BY customer_id
)
SELECT customer_id, total,
       DENSE_RANK() OVER (ORDER BY total DESC) AS rnk
FROM spend
QUALIFY rnk <= 3;   -- Snowflake
```

❄ **Standard SQL:** wrap in a subquery and filter `WHERE rnk <= 3`.

---

97. Recursive CTE — the full management chain (org hierarchy)

```sql
-- Start at the CEO, then repeatedly join to find each person's reports
WITH RECURSIVE org AS (
    SELECT emp_id, name, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL          -- anchor: the top
    UNION ALL
    SELECT e.emp_id, e.name, e.manager_id, o.level + 1
    FROM employees e
    JOIN org o ON e.manager_id = o.emp_id   -- recursive step
)
SELECT level, name FROM org ORDER BY level, name;
```

❄ **Snowflake:** identical (`WITH RECURSIVE` supported). 💬 Recursion = anchor (starting rows) + recursive member (rows built from the previous step) until nothing new is added. The tool for trees/hierarchies.

---

98. Recursive CTE — generate a number series 1..10

```sql
WITH RECURSIVE nums AS (
    SELECT 1 AS n
    UNION ALL
    SELECT n + 1 FROM nums WHERE n < 10
)
SELECT n FROM nums;
```

❄ **Snowflake:** identical. Useful for building calendars/date spines to fill gaps.

---

99. Fill missing months with zero revenue (date spine + LEFT JOIN)

```sql
WITH RECURSIVE months AS (
    SELECT DATE '2023-06-01' AS m
    UNION ALL
    SELECT DATEADD('month', 1, m) FROM months WHERE m < DATE '2023-08-01'
)
SELECT m,
       COALESCE(SUM(o.amount), 0) AS revenue
FROM months
LEFT JOIN orders o ON DATE_TRUNC('month', o.order_date) = m
GROUP BY m
ORDER BY m;
```

❄ **Snowflake:** identical. 💬 Reporting must show months with **zero** activity — a generated date spine + `LEFT JOIN` + `COALESCE` does it. Common real-world requirement.

---

100. Year-over-year style growth % per month (LAG + calculation) — Capstone

```sql
WITH monthly AS (
    SELECT DATE_TRUNC('month', order_date) AS mth, SUM(amount) AS revenue
    FROM orders GROUP BY 1
)
SELECT mth, revenue,
       LAG(revenue) OVER (ORDER BY mth) AS prev,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY mth))
             / NULLIF(LAG(revenue) OVER (ORDER BY mth), 0), 1) AS growth_pct
FROM monthly
ORDER BY mth;
```

❄️ **Snowflake:** identical. 💬 `NULLIF(x, 0)` guards against divide-by-zero when the previous month is 0/NULL. This single query combines CTEs, window functions, and safe math — everything from this tier.

🎓 You Finished All 100!

You've now practised every pattern SQL interviews throw at you: joins, subqueries, CTEs, all window functions, ranking, running totals, LAG/LEAD, gaps-and-islands, pivots, and recursion — in both standard SQL and Snowflake.

Next steps: - Redo every problem you needed help on, one week later. - Try the same patterns on a dataset *you* care about. - Put "solved 100 SQL problems incl. window functions & recursion in SQL + Snowflake" on your resume.

⬅️ Back to Easy · Medium · Problem Index